STAFF REVIEW · H446 016 source models

Separate from the eight-page pupil workbook. This copy is in a public repository and is NOT access controlled. Check technical correctness before use.

Printed question mapping: Booklet Q1 = original online Q1; Booklet Q2 = original online Q4

Do Now — source answers

1. true.

2. 3 — the number of rows.

3. Argument: 7. Parameter: number.

4. An outer loop visits rows 0–2; an inner loop visits columns 0–2. The same operation is reused instead of duplicated, making the code easier to maintain.

I Do — source worked steps

1. Use nested loops — CODE:
```
for (int row = 0; row < board.GetLength(0); row++)
{
for (int column = 0; column < board.GetLength(1); column++)
{
}
}
```

2. One free square means not full — CODE:
```
if (board[row, column] != "X" &&
board[row, column] != "O")
{
return false;
```
} — The function can stop immediately when it finds a free square.

3. No free square was found — CODE:
```
return true;
```
— This line is reached only after every position has been checked.

4. Check draw after checking win — CODE:
```
if (HasWon(currentPlayer))
{
// win
}
else if (BoardFull())
{
// draw
}
```

We Do — guided CHECK

1. Complete the condition
CODE:
```
if (board[0,0] == player &&
```

```
board[1,1] == player &&
board[2,2] == player)
{
return true;
}
```

2. Build the Boolean test

3. Complete the sequence

4. Predict the functions

You Do — source mark schemes

1. HasWon()
CODE:

```
public static bool HasWon(string player)
{
return
(board[0,0] == player && board[0,1] == player && board[0,2] == player) ||
(board[1,0] == player && board[1,1] == player && board[1,2] == player) ||
(board[2,0] == player && board[2,1] == player && board[2,2] == player) ||

(board[0,0] == player && board[1,0] == player && board[2,0] == player) ||
(board[0,1] == player && board[1,1] == player && board[2,1] == player) ||
(board[0,2] == player && board[1,2] == player && board[2,2] == player) ||

(board[0,0] == player && board[1,1] == player && board[2,2] == player) ||
(board[0,2] == player && board[1,1] == player && board[2,0] == player);
}
```

Check: 3 rows + 3 columns + 2 diagonals = 8 possible winning lines.

2. BoardFull()
CODE:

```
public static bool BoardFull()
{
for (int row = 0; row < board.GetLength(0); row++)
{
for (int column = 0; column < board.GetLength(1); column++)
{
if (board[row, column] != "X" &&
board[row, column] != "O")
{
return false;
}
}
}

return true;
}
```

Logic: one free square is enough to return false . Only return true after every square has been checked.

3. Main() game loop
CODE:

```
public static void Main()
{
string currentPlayer = "X";
bool gameOver = false;

while (gameOver == false)
{
DrawBoard();

Console.Write("Player " + currentPlayer + ", choose 1-9: ");
int move = int.Parse(Console.ReadLine());

if (MakeMove(move, currentPlayer))
{
if (HasWon(currentPlayer))
{
DrawBoard();
Console.WriteLine("Player " + currentPlayer + " wins!");
gameOver = true;
}
else if (BoardFull())
{
DrawBoard();
Console.WriteLine("Draw!");
gameOver = true;
}
else
{
currentPlayer = SwitchPlayer(currentPlayer);
}
}
else
{
Console.WriteLine("Invalid move - try again.");
}
}
}
```

Order matters: check win first, then draw, then switch player.

4. Complete playable version
CODE:

```
using System;
```

```
public class Program
{
public static string[,] board =
{
{ "1", "2", "3" },
{ "4", "5", "6" },
{ "7", "8", "9" }
};

public static void Main()
{
string currentPlayer = "X";
bool gameOver = false;

while (gameOver == false)
{
DrawBoard();

Console.Write("Player " + currentPlayer + ", choose 1-9: ");
int move = int.Parse(Console.ReadLine());

if (MakeMove(move, currentPlayer))
{
if (HasWon(currentPlayer))
{
DrawBoard();
Console.WriteLine("Player " + currentPlayer + " wins!");
gameOver = true;
}
else if (BoardFull())
{
DrawBoard();
Console.WriteLine("Draw!");
gameOver = true;
}
else
{
currentPlayer = SwitchPlayer(currentPlayer);
}
}
else
{
Console.WriteLine("Invalid move - try again.");
}
}
}
```

```
public static void DrawBoard()
{
Console.WriteLine();

for (int row = 0; row < board.GetLength(0); row++)
{
for (int column = 0; column < board.GetLength(1); column++)
{
Console.Write(board[row, column]);

if (column < board.GetLength(1) - 1)
{
Console.Write(" | ");
}
}

Console.WriteLine();

if (row < board.GetLength(0) - 1)
{
Console.WriteLine("--+---+--");
}
}

Console.WriteLine();
}

public static bool MakeMove(int move, string player)
{
if (move < 1 || move > 9)
{
return false;
}

int row = (move - 1) / 3;
int column = (move - 1) % 3;

if (board[row, column] == "X" ||
board[row, column] == "O")
{
return false;
}

board[row, column] = player;
return true;
}
```

```
public static string SwitchPlayer(string player)
{
if (player == "X")
{
return "O";
}
else
{
return "X";
}
}

public static bool HasWon(string player)
{
return
(board[0,0] == player && board[0,1] == player && board[0,2] == player) ||
(board[1,0] == player && board[1,1] == player && board[1,2] == player) ||
(board[2,0] == player && board[2,1] == player && board[2,2] == player) ||

(board[0,0] == player && board[1,0] == player && board[2,0] == player) ||
(board[0,1] == player && board[1,1] == player && board[2,1] == player) ||
(board[0,2] == player && board[1,2] == player && board[2,2] == player) ||

(board[0,0] == player && board[1,1] == player && board[2,2] == player) ||
(board[0,2] == player && board[1,1] == player && board[2,0] == player);
}

public static bool BoardFull()
{
for (int row = 0; row < board.GetLength(0); row++)
{
for (int column = 0; column < board.GetLength(1); column++)
{
if (board[row, column] != "X" &&
board[row, column] != "O")
{
return false;
}
}
}

return true;
}
}
```

Final test: play more than once and deliberately test invalid moves, each type of win and a draw.

Exit Ticket — source answer

1. True. After a valid move, check for a win before checking whether the board is full and the game is a draw.